

NAME: KRISHNAPRIYA P

REG NO:192511279

COURSE CODE: CSA0613

COURSE NAME : Design

Analysis of Algorithm

CO4 – AT3 --Comparative Analysis

Q20. Bellman-Ford vs Dijkstra (DP vs Greedy)

Given a weighted graph with negative edges, compute shortest paths using Bellman-Ford and Dijkstra. Compare correctness and performance.

Analyze why Dijkstra fails for negative weights. Evaluate complexity differences. Discuss real-world applications and generate insights on shortest path algorithm selection.

1. Given Weighted Graph

Consider the following directed weighted graph with source vertex

A:

Edge	From → To	Weight
1	A → B	4
2	A → C	5
3	B → C	-3
4	B → D	6
5	C →	4

The graph contains a negative edge $B \rightarrow C = -3$. There is no negative-weight cycle in this example.

2. Bellman-Ford Using Dynamic Programming

Bellman-Ford repeatedly relaxes all edges. It can correctly handle negative edge weights as long as there is no reachable negative-weight cycle.

Step 1: Initialize Distances

- $\text{Distance}(A) = 0$
- $\text{Distance}(B) = \infty$
- $\text{Distance}(C) = \infty$
- $\text{Distance}(D) = \infty$

Step 2: Relax the Edges

Starting from A:

$A \rightarrow B$

- $0 + 4 = 4$
- $\text{Distance}(B) = 4$

$A \rightarrow C$

- $0 + 5 = 5$
- $\text{Distance}(C) = 5$

$B \rightarrow C$

- $4 + (-3) = 1$
- $\text{Distance}(C)$ becomes 1

$B \rightarrow D$

- $4 + 6 = 10$
- $\text{Distance}(D) = 10$

$C \rightarrow D$

- $1 + 4 = 5$
- $\text{Distance}(D)$ becomes 5

After further passes, no distance can be improved.

3. Dijkstra Using Greedy Method

Dijkstra's algorithm selects the unvisited vertex having the smallest tentative distance and permanently fixes its distance.

Step 1: Initialize

- $\text{Distance}(A) = 0$
- $\text{Distance}(B) = \infty$
- $\text{Distance}(C) = \infty$
- $\text{Distance}(D) = \infty$

Step 2: Process A

From A:

- $A \rightarrow B = 4$
- $A \rightarrow C = 5$

Therefore:

- $\text{Distance}(B) = 4$
- $\text{Distance}(C) = 5$

Step 3: Process B

B has the smallest tentative distance of 4.

For B → C:

- $4 + (-3) = 1$
- Distance(C) becomes 1

For B → D:

- $4 + 6 = 10$
- Distance(D) = 10

Step 4: Process C

C now has distance 1.

For C → D:

- $1 + 4 = 5$
- Distance(D) becomes 5

Final Result

For this particular graph, Dijkstra can also obtain:

Vertex	Distance
A	0
B	4
C	1
D	5

4. Why Does Dijkstra Fail for Negative Weights?

Dijkstra follows a greedy assumption: once a vertex with the smallest tentative distance is selected, its distance is considered final. This assumption fails when negative edges are present.

Example

Consider:

Edge	Weight
$A \rightarrow B$	2
$A \rightarrow C$	5
$C \rightarrow B$	-6

Dijkstra initially obtains:

- $\text{Distance}(B) = 2$
- $\text{Distance}(C) = 5$

Since B has the smaller distance, Dijkstra may permanently select B with distance 2.

However, another path exists:

$A \rightarrow C \rightarrow B$

Its cost is:

$$5 + (-6) = -1$$

Therefore, the actual shortest distance to B is -1, not 2. The problem is that Dijkstra has already finalized B before discovering the better path through the negative edge.

- Dijkstra depends on non-negative edge weights.
- Negative edges can invalidate its greedy decision.
- Bellman-Ford can reconsider distances through repeated relaxation.
- Hence, Bellman-Ford is the safer choice when negative weights

are present.

6. Computational Complexity

Bellman-Ford

Let:

- V = number of vertices
- E = number of edges

Time Complexity:

$O(VE)$

Bellman-Ford relaxes all edges up to $V - 1$ times.

Space Complexity:

$O(V)$

Dijkstra

Using a simple array:

Time Complexity = $O(V^2)$

Using an adjacency list with a binary heap:

Time Complexity = $O((V + E) \log V)$

For connected sparse graphs, this is commonly expressed as:

$O(E \log V)$

Space Complexity = $O(V + E)$ when an adjacency list and priority queue are used.

7. Dynamic Programming vs Greedy Strategy

Bellman-Ford – Dynamic Programming

Bellman-Ford progressively improves shortest-path estimates by relaxing

edges repeatedly. A distance that was previously large can be reduced when a better path is discovered through another vertex. This makes Bellman-Ford suitable for graphs containing negative edges.

Dijkstra – Greedy

Dijkstra always chooses the vertex with the smallest current distance. It permanently accepts this choice because, with non-negative edges, no later path can make that distance smaller. Therefore, the greedy strategy is efficient but depends on the important condition that all edge weights must be non-negative.

8. Real-World Applications

Bellman-Ford Applications

- Network routing where edge costs may change or become negative.
- Financial and economic models involving gains and losses.
- Graph problems requiring negative-cycle detection.
- Systems where correctness with negative edge values is more important than speed.

Dijkstra Applications

- GPS and road-network navigation.
- Computer network routing with non-negative link costs.
- Robot navigation and path planning.
- Transportation and map-based applications.
- Large-scale shortest-path problems requiring fast computation.

9. Critical Evaluation

Bellman-Ford Strengths

- Correctly handles negative edge weights.
- Can detect reachable negative-weight cycles.
- Simple edge-relaxation approach.
- More general than Dijkstra for weighted graphs.

Bellman-Ford Limitations

- Higher time complexity of $O(VE)$.
- Slower for large graphs.
- Requires repeated relaxation of edges.

Dijkstra Strengths

- Faster for graphs with non-negative weights.
- Efficient greedy approach.
- Works well with priority queues.
- Suitable for large road and network graphs.

Dijkstra Limitations

- Standard Dijkstra cannot safely handle negative edge weights.
- Greedy choices can become incorrect when negative edges exist.
- It does not provide Bellman-Ford's negative-cycle detection capability.

10. Insights for Shortest-Path Algorithm Selection

The choice of algorithm should depend mainly on the properties of the graph.

Use Bellman-Ford When:

- The graph contains negative edge weights.

- Negative-weight cycle detection is required.
- Correctness is more important than execution speed.
- The graph is relatively small or moderate in size.

Use Dijkstra When:

- All edge weights are non-negative.
- Fast shortest-path computation is required.
- The graph is large and sparse.
- The application involves road distances, travel times, or other non-negative costs.

Selection Rule

- Negative edge weights → Bellman-Ford
- Non-negative edge weights + high performance → Dijkstra
- Negative-cycle detection → Bellman-Ford
- Large road/network graph with non-negative costs → Dijkstra

Conclusion

Bellman-Ford and Dijkstra both solve the single-source shortest-path problem, but they use different approaches. Bellman-Ford uses a dynamic-programming style of repeated relaxation and can correctly handle negative edge weights, whereas Dijkstra uses a greedy strategy that depends on all edge weights being non-negative.

For the given graph, Bellman-Ford produces the shortest distances:

- $A = 0$
- $B = 4$
- $C = 1$

- **D = 5**

Although Dijkstra also obtains the correct result for this particular graph, it is not guaranteed to be correct when negative edges are present. Therefore, Bellman-Ford should be selected when negative edge weights may occur, while Dijkstra is preferred for faster computation when all edge weights are non-negative.

